

SIMATS ENGINEERING

ASSESSMENT TOOL 2 : Scenario-Based

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA15

COURSE OUTCOME COVERED

CO2: *Analyze virtualization architectures to identify performance and security issues in cloud computing environments. (BL3)*

Dave's Taxonomy: Manipulation (Level 2)
Question Count: 5

Total Marks: 25

Code of Conduct

I B h u v a n k u m a r J (192524040)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Bhuvan kumar J

192524040

Assessment Tasks

Scenario-Based Questions

1. A media company wants to migrate its video streaming platform to the cloud. The system must handle millions of concurrent users with minimal buffering. Design a cloud architecture using scalability and caching techniques. Explain how load balancing and auto-scaling can ensure smooth streaming. Discuss cost optimization strategies for such high-demand workloads.

Video Streaming Platform Migration (Scalability & Caching)

Architecture Design

Global Content Delivery Layer

- Use a **CDN** (CloudFront, Akamai, or Cloudflare) as the first line of defense — it caches video segments (using adaptive bitrate formats like HLS/DASH) at edge locations close to users, drastically cutting latency and origin load.
- Store master video files in **object storage** (S3, Azure Blob, GCS) with lifecycle policies to move older/less-watched content to cooler storage tiers.

Transcoding & Processing

- Use an **event-driven transcoding pipeline**: upload triggers a serverless function (Lambda/Cloud Functions) that kicks off a managed transcoding service (AWS Elemental MediaConvert, Azure Media Services) to produce multiple bitrate/resolution renditions for adaptive streaming.

Compute Layer

- Stateless microservices (user auth, recommendations, metadata, billing) run in containers on Kubernetes (EKS/GKE/AKS) or a managed serverless platform, enabling independent scaling per service.

Caching Strategy (multi-layer)

- **Edge/CDN cache** — video segments, thumbnails, static assets.
- **In-memory cache** (Redis/Memcached) — session data, user profiles, trending/recommendation results, API responses.
- **Database read replicas + query cache** — reduces load on the primary datastore for metadata lookups (titles, catalogs, watch history).
- Use **cache invalidation policies** (TTL + event-based purge) so new releases or content takedowns propagate quickly.

Load Balancing & Auto-Scaling for Smooth Streaming

- **Global (DNS-based) load balancing** (Route 53, Azure Traffic Manager) routes users to the nearest healthy region.
- **Regional load balancers** (ALB/NLB, Azure Load Balancer) distribute traffic across container/VM instances, performing health checks and removing unhealthy nodes automatically.
- **Auto-scaling groups / Horizontal Pod Autoscalers** monitor CPU, memory, request latency, and concurrent connections, scaling compute up during peak hours (e.g., a major sports final or new season release) and down during quiet periods.
- **Predictive scaling** can be layered on top using historical traffic patterns (e.g., scale up before a known prime-time surge) combined with reactive scaling for unexpected spikes.
- Because video delivery itself is offloaded to the CDN, origin servers mainly handle API/control-

plane traffic, which is far easier to scale than raw video egress.

Cost Optimization Strategies

- **CDN + caching** reduces expensive repeated origin egress — the single largest cost driver in streaming.
 - **Tiered storage**: hot storage for new/popular content, cold/archive storage for rarely watched back-catalog titles.
 - **Spot/preemptible instances** for non-critical, interruptible workloads like batch transcoding.
 - **Reserved/committed-use instances** for baseline, predictable capacity; on-demand or spot for burst capacity.
 - **Right-sizing** compute resources continuously based on utilization metrics.
 - **Compression and efficient codecs** (AV1, H.265) to reduce bandwidth and storage costs.
 - **Multi-CDN strategy** to negotiate better rates and avoid vendor lock-in on egress pricing.
-

2. A fintech startup is deploying applications using containers across multiple cloud providers. They face challenges in maintaining consistency and managing deployments. Propose a solution using container orchestration and multi-cloud strategies. Explain how service discovery and scaling can be handled efficiently. Highlight the benefits and risks of a multi-cloud deployment model.

Fintech Startup — Multi-Cloud Container Deployment

Proposed Solution

Container Orchestration

- Adopt **Kubernetes** as the common orchestration layer across providers (EKS, AKS, GKE, or self-managed) since it provides a consistent API regardless of the underlying cloud.
- Use a **GitOps approach** (ArgoCD/Flux) so deployments are declarative and version-controlled, ensuring consistency: the same manifests/Helm charts deploy identically across clusters.
- Standardize on **Infrastructure as Code** (Terraform) to provision clusters and supporting infrastructure identically across clouds.

Multi-Cloud Strategy

- Use a **multi-cloud management platform** (Anthos, Azure Arc, or Rancher) to provide a single control plane visibility over clusters in different providers.
- Maintain a **shared container registry** (or registry mirroring) so images are consistent and don't need to be rebuilt per cloud.
- Use **abstraction layers** for cloud-specific services (e.g., an internal API wrapper around each provider's managed database/secrets manager) to avoid tight coupling to one vendor's APIs.

Service Discovery & Scaling

- **Service mesh** (Istio/Linkerd/Consul Connect) handles service discovery, mutual TLS, traffic routing, and observability consistently across clusters — critical when services in one cloud need to call services in another.
- **Kubernetes-native service discovery** (DNS-based, via CoreDNS) works within each cluster; for cross-cluster discovery, a mesh with multi-cluster support (Istio multi-cluster) or a global API gateway ties them together.

- **Horizontal Pod Autoscaler (HPA)** and **Cluster Autoscaler** independently scale each cluster based on load; a **global load balancer** in front distributes traffic across clouds based on latency, health, and capacity.

Benefits and Risks of Multi-Cloud

Benefits

- Avoids vendor lock-in and gives negotiating leverage.
- Improves resilience — an outage in one provider doesn't take the whole platform down.
- Allows choosing best-of-breed services per cloud (e.g., one provider's ML tools, another's compliance certifications).
- Useful for meeting data residency/regulatory requirements common in fintech.

Risks

- Increased **operational complexity** — different networking models, IAM systems, and monitoring tools per cloud.
- **Higher cost** from data egress between clouds and duplicated tooling.
- **Security surface area** grows — more identities, more APIs, more points of failure to secure and audit (a major concern for a regulated fintech).
- **Skill and tooling overhead** — teams need expertise across multiple platforms.
- **Latency** for cross-cloud service calls if not architected carefully.

3. An e-learning platform experiences data loss due to accidental deletion in the cloud.
The organization needs a robust backup and disaster recovery strategy.
Design a solution using cloud-native backup, replication, and versioning.

E-Learning Platform — Backup & Disaster Recovery

Solution Design

Backup Strategy

- **Automated, scheduled backups** of databases and file storage (e.g., AWS Backup, Azure Backup) with configurable frequency based on data criticality (e.g., hourly for transactional data, daily for content).
- **Versioning** enabled on object storage (S3 Versioning) so accidental deletions or overwrites can be rolled back to a prior object version.
- **Immutable/WORM backups** or backup vault lock to protect against ransomware or accidental/malicious deletion of the backups themselves.
- **Cross-region replication** of both primary data and backups so a regional failure doesn't destroy the only copies.

Replication

- Database replication (synchronous within a region for low RPO, asynchronous cross-region for DR) using managed services (RDS Multi-AZ + cross-region read replica, Azure SQL geo-replication).
- Storage replication for uploaded content (videos, course materials) across regions.

RTO and RPO

- **RPO (Recovery Point Objective)** — determined by backup/replication frequency. Continuous or

near-real-time replication (synchronous within-region, asynchronous cross-region every few minutes) achieves a low RPO, minimizing acceptable data loss (e.g., RPO of a few minutes rather than a full day).

- **RTO (Recovery Time Objective)** — determined by how quickly systems can be brought back online. A **warm standby** (scaled-down but running environment in a second region) achieves a much lower RTO than restoring entirely from cold backups, at extra cost. A **pilot light** approach (minimal core infra always running, scaled up on failover) is a middle ground.
- The organization should define target RTO/RPO per system tier (e.g., core LMS = minutes; archived content = hours) rather than one blanket target, since low RTO/RPO is expensive.

Steps for Business Continuity

1. Classify data/systems by criticality and assign RTO/RPO targets to each.
 2. Automate backups with versioning and cross-region replication.
 3. Maintain a documented, tested **disaster recovery runbook** (failover steps, roles, communication plan).
 4. Run **regular DR drills/game days** to validate that failover actually meets the targeted RTO/RPO.
 5. Implement **monitoring and alerting** on backup job success/failure.
 6. Use **Infrastructure as Code** so a full environment can be redeployed quickly in a new region if needed.
 7. Maintain an **incident communication plan** to keep students/instructors informed during an outage.
-

4. A government agency is adopting a hybrid cloud model for sensitive and non-sensitive data. Some applications must remain on-premises due to compliance requirements. Design a hybrid architecture that ensures secure communication between environments. Explain how identity management and encryption should be implemented. Discuss challenges in managing hybrid cloud environments.

Government Agency — Hybrid Cloud Model

Architecture Design

- **On-premises data center** hosts sensitive/regulated workloads (citizen PII, classified data) that cannot legally or operationally move to the public cloud.
- **Public cloud** hosts non-sensitive, elastic workloads (public-facing portals, non-sensitive analytics, dev/test environments).
- Connect the two via a **dedicated, private network link** (AWS Direct Connect, Azure ExpressRoute, or a site-to-site VPN as a lower-cost/backup option) rather than routing sensitive traffic over the public internet.
- Use a **hybrid API gateway** or integration layer so cloud-hosted applications can securely call on-premises services (and vice versa) without exposing internal systems directly to the internet.
- Segment the network using **VPC/VNet with private subnets**, firewalls, and network security groups mirroring on-prem segmentation policies.

Identity Management and Encryption

- **Federated identity** — extend the on-prem directory (Active Directory) into the cloud via identity federation (AD FS, Azure AD Connect, or SAML/OIDC) so a single identity system governs access on both sides, with **multi-factor authentication** enforced everywhere.
- Apply **role-based access control (RBAC)** and the **principle of least privilege** consistently across environments.
- **Encryption in transit** — TLS for all communication, and IPsec/VPN tunneling over the dedicated link as an added layer.
- **Encryption at rest** — cloud-native key management (AWS KMS, Azure Key Vault) for cloud data; hardware security modules (HSMs) or on-prem key management for the most sensitive on-prem data, with keys never leaving agency control where compliance mandates it.
- Maintain **audit logging** of all cross-environment access for compliance reporting.

Challenges in Managing Hybrid Cloud

- **Consistency** — keeping security policies, patching, and configurations aligned across two very different environments.
 - **Latency** between on-prem and cloud components for tightly coupled workloads.
 - **Complex identity federation** — synchronization issues, trust relationship management.
 - **Compliance mapping** — proving to auditors that data never leaves approved boundaries.
 - **Skill gaps** — staff need expertise in both traditional on-prem infrastructure and cloud-native tooling.
 - **Cost visibility** — harder to get a unified view of spend and utilization across both environments.
 - **Tooling fragmentation** — monitoring, logging, and management tools often differ between on-prem and cloud, complicating a unified operational view.
-

5. A SaaS provider needs to ensure high availability and fault tolerance for its application.
The system must continue operating even if one region goes down.
Propose a multi-region deployment strategy in the cloud.
Explain how data replication and failover mechanisms should be configured.
Discuss monitoring and alerting strategies for proactive issue detection.

SaaS Provider — High Availability & Fault Tolerance

Multi-Region Deployment Strategy

- Deploy the application stack (compute, database, cache) identically in **at least two geographically separate regions**, using Infrastructure as Code for consistency.
- Use an **active-active** model (both regions serving live traffic, for maximum availability and load distribution) or **active-passive** (a standby region that takes over on failure, simpler but with a brief failover window) depending on cost tolerance and consistency requirements.
- Front the regions with a **global load balancer/DNS routing** (Route 53 with health checks, Azure Front Door, Google Cloud Load Balancing) that directs users to the nearest healthy region and automatically reroutes traffic if a region becomes unhealthy.

Data Replication and Failover

- Use a **globally distributed database** (DynamoDB Global Tables, Cosmos DB, Aurora Global Database, Spanner) or configure **cross-region replication** for the chosen database, balancing consistency vs. latency (synchronous replication gives stronger consistency but higher latency; asynchronous is faster but risks small data loss on failover).
- Implement **automated failover** using health checks that detect a region-wide outage and redirect DNS/load-balancer traffic within seconds to minutes.
- Regularly test failover through **chaos engineering practices** (e.g., simulating a region outage) to ensure the mechanism actually works under real conditions, not just on paper.
- Keep **stateless application tiers** so any instance in any region can serve any request, simplifying failover.

Monitoring and Alerting

- Centralized **observability stack** (Prometheus/Grafana, Datadog, CloudWatch, Azure Monitor) aggregating metrics, logs, and traces from all regions into one view.
- **Synthetic monitoring/health checks** continuously probe endpoints from multiple locations to detect degradation before users report it.
- **Distributed tracing** (OpenTelemetry, X-Ray) to pinpoint which service/region is causing latency or errors in a multi-region setup.
- Define **SLOs/SLIs** (e.g., 99.95% availability, p99 latency thresholds) and alert when metrics approach the error budget threshold — not just on hard failures.
- **Escalation policies and on-call rotations** (PagerDuty, Opsgenie) ensure alerts reach the right engineers quickly, with runbooks for common failure scenarios to speed up resolution.

Scenario-Based Rubric

Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Understanding of Scenario	20%	Demonstrates complete understanding of the scenario context	Good understanding with minor gaps	Partial understanding	Fails to understand the scenario
Identification of Risks / Issues	20%	Identifies all major issues correctly	Identifies most issues	Limited issue identification	Misses major issues
Application of Concepts	25%	Applies virtualization concepts effectively	Applies concepts with minor mistakes	Partially correct application	Weak application
Decision Justification	20%	Decisions are well justified and technically strong	Reasonable justification	Weak justification	No useful justification
Clarity	15%	Clear and well-organized response	Generally clear	Somewhat unclear	Poorly presented